

Poster: Efficient Threshold ML-DSA up to 6 Parties

Sofia Celi
Brave Research & University of
Bristol
Bristol, UK

Rafael del Pino
PQShield SAS
Paris, France

Thomas Espitau
PQShield SAS
Paris, France

Guilhem Niot
PQShield SAS & Univ Rennes, CNRS,
IRISA
Rennes, France

Thomas Prest
PQShield SAS
Paris, France

Abstract

Threshold signature schemes enable a group of users to collaboratively produce digital signatures without revealing any individual share. With the current NIST post-quantum standardization underway, the lack of efficient and practical threshold variants of standardized schemes hinders adoption. We introduce the first threshold signature scheme compatible with the ML-DSA standard (Module-Lattice-based Digital Signature Algorithm), supporting up to 6 parties, while retaining efficient signing. Our work uses advanced short secret sharing techniques and optimized rejection sampling to balance communication and correctness in distributed settings. We implement our construction in Go and benchmark it in local, LAN, and WAN deployments. Results show that our threshold ML-DSA is both practical and compatible with real-world applications such as multi-device cryptocurrency wallets, threshold TLS, and Tor's directory authorities.

CCS Concepts

• Security and privacy → Digital signatures; Key management.

Keywords

Threshold signatures, lattice-based cryptography, ML-DSA

ACM Reference Format:

Sofia Celi, Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. 2025. Poster: Efficient Threshold ML-DSA up to 6 Parties. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei, Taiwan. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3719027.3760739>

1 Motivation

As the cryptographic community and real-world systems transition toward post-quantum secure standards, deploying quantum-safe threshold cryptography remains an unmet challenge. Real-world systems, including secure financial infrastructures, content delivery networks, and privacy-preserving protocols, rely on this type of multi-party signatures to increase resilience, provide privacy

and distribute trust. However, most threshold signature schemes are either (i) not quantum-safe, (ii) incompatible with standardized post-quantum primitives [2, 5, 7, 9], or (iii) too inefficient or costly for deployment [3]. Most notably, ML-DSA, one of the NIST-standardized post-quantum signature and one that is being widely deployed, offers compactness and simplicity, but no known threshold construction prior to this work could maintain compatibility with its verification procedure. This difficulty arises from the difficulty of efficiently distributing its rejection-sampling-based signing algorithm across multiple parties, which includes the sampling of short vectors as well as a specific rounding mechanism.

From a practical perspective, threshold signing over the post-quantum standardised scheme ML-DSA must address:

- The complexity of distributing rejection sampling across several parties.
- Need for low computational and communication overhead.

2 Contribution

In the light of this motivation, our primary contributions (to be presented as a poster) include:

- **First ML-DSA-Compatible Threshold Signature Scheme.** We design and implement the first practical threshold signature scheme compatible with ML-DSA, supporting up to $N = 6$ parties and any threshold $T \leq N$. Our scheme efficiently supports all ML-DSA parameter sets, targeting NIST security levels 1, 3, and 5.
- **Short Secret Sharing for Lattices.** Using the short replicated secret sharing technique of [4], we distribute secrets as low-norm shares, enabling per-party rejection sampling. While this lowers the overall success probability per signing attempt, it is a key enabler for our efficient thresholdization.
- **Optimized Parallel Rejection Sampling.** We employ rejection sampling based on hyperballs, unlike standard ML-DSA signing which relies on hypercubes. In high-dimension, the lower difference between the target and the real distributions using hyperballs provides a valuable boost of the protocol success probability, and balances the degradation of per-party rejection sampling. Combined with parallel instances and an optimized distribution of shares among parties, this enables practical threshold signing even as the number of parties increases.
- **Formal Security and Correctness.** We formally prove that our scheme retains the unforgeability guarantees of ML-DSA

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CCS '25, Taipei, Taiwan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1525-9/2025/10

<https://doi.org/10.1145/3719027.3760739>

under the same MLWE assumptions. We include concrete bounds for correctness and liveness in the presence of retries.

- **Full Implementation and Benchmarks.** Our Go implementation extends the CIRCL library and is benchmarked in LAN, WAN, and local settings. For ML-DSA-44, targeting NIST level I, communication costs range from 10.5 kB to 525 kB per party, and protocol runtimes are a few milliseconds. Even in WAN deployments across continents, signing latency remains under 1s.
- **Real-World Applications.** Our scheme enables threshold TLS in CDNs with keyless operations, multi-device wallets with high availability, and secure randomness beacons for Tor directory authorities.

3 Main Construction

Our threshold variant of ML-DSA is tailored for small threshold settings, building on Finally! [6], a recent three-round scheme with compact signatures. While Finally! achieves significant improvements over prior work, its signatures remain larger than those of ML-DSA for few parties. Our key insight is that by reintroducing errors in the commitment w of ML-DSA, we can safely reveal commitments without rounding—even in case of signature rejection—by leveraging techniques from [6]. Through targeted optimizations and parameter tuning, we obtain a practical threshold ML-DSA that securely distributes signing and further improves on Finally! by trading communication for higher signature capacity, enabling smaller parameters while retaining compatibility.

The protocol samples several short secrets and aggregates their commitments to form the public key. These secrets are shared so that any subset of at least T parties can reconstruct them. Signing proceeds via parallel rejection sampling, performed independently by each party. This per-party rejection step leads to a lower per-iteration success probability, which we address by employing tighter, non-uniform distributions for the rejection step—specifically, unbalanced hyperballs—which are especially effective in small-threshold regimes. This approach exploits the slack between ML-DSA’s uniform sampling and the more efficient hyperball-based sampling, allowing us to maintain compact signatures in the distributed setting.

To further boost protocol liveness, we run $K > 1$ parallel signing attempts: each party produces K commitments and responses, increasing the chance of a successful signing round. Our construction thus balances efficiency, correctness, and practicality, making threshold ML-DSA suitable for real-world deployment without sacrificing signature size or security.

4 Evaluation

Implementation. We fully implemented our construction to ease integration with other systems and libraries. We chose GoLang for its popularity in cryptographic implementations and low incidence of security issues [1, 11]. Our implementation of Threshold ML-DSA builds on the code of the CIRCL library [8].

4.1 Experimental Analysis

We evaluated the performance of Threshold ML-DSA locally, as well as in LAN and WAN settings. All parties ran on a consumer-grade

Table 1: Local simulation for the average (mean) costs of Threshold ML-DSA-44 All costs are computed on a MacOS M3 machine and grouped by N . **Green** and **light green** indicate the most and second-most optimal cases per group (N) and per scheme, where applicable.

(T, N)	KeyGen (ms)	Sign+Combine (ms)	Verify (ms)
Threshold ML-DSA			
(3,3)	0.3669	0.6810	0.0306
(2,4)	0.1709	0.4570	
(3,4)	0.2062	1.0961	
(4,4)	0.1655	1.3672	
(3,5)	0.2870	2.2263	
(4,5)	0.2940	4.9832	
(5,5)	0.1956	2.8453	
(4,6)	0.5016	12.1949	
(6,6)	0.2181	7.6784	

MacBook M3 for local and LAN experiments with 25 GB RAM. We conducted single-threaded local simulations of both schemes to benchmark key generation, signing (across parallel rounds), and verification. Each party’s computation was emulated in a single process, capturing fine-grained timing data for each protocol phase. The runtime (i.e., computational cost) for various (T, N) values is shown in Table 1, for Threshold ML-DSA compatible with the parameter set 44 of ML-DSA. Visualizations of bandwidth and runtime for Threshold ML-DSA-44 appear in Fig. 1.

REMARK. For clarity, in this version we only display benchmarks for ML-DSA-44, which targets NIST security level 1. Results for other parameter sets and security levels are similarly efficient.

For our WAN and LAN experiments, we implemented the communication layer of our schemes using libp2p [12], a modular peer-to-peer networking stack. Each party acts as an independent libp2p peer, with its identity embedded as its network address, and communicates via a custom protocol over a direct TCP stream. Parties listen for incoming connections on a specified TCP port, and others connect using multiaddress. After establishing a connection, they exchange messages over a persistent bidirectional libp2p stream. This setup ensures execution over a realistic asynchronous, decentralized network, with libp2p handling connection management, peer identity, and stream multiplexing. NAT traversal and relaying were avoided for simplicity, assuming direct TCP connectivity.

LAN experiment. We present our results in Table 2. Protocol instances run on separate machines over the same Wi-Fi network. Latency was estimated using an RTT over a TCP connection, yielding 124.166 μ s.

WAN experiment. We present our results in Table 3 for Threshold ML-DSA using a mesh network topology. The protocol ran on Amazon EC2 “t2.small” instances (Ubuntu 24.04.02, 2.0 GiB RAM), except for a MacOS M3 machine in Taipei, Taiwan, acting as “leader” and initiating connections. Other machines were distributed across Virginia (USA), London (UK), and Seoul (South

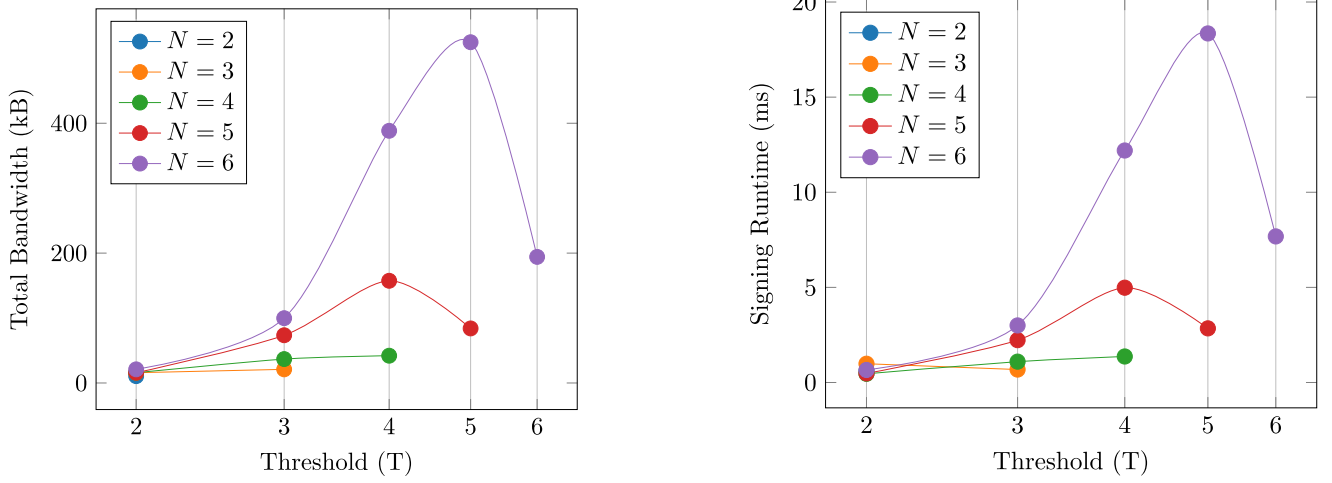


Figure 1: Bandwidth costs (left) and runtime signing costs (right) for Threshold ML-DSA-44. Note that the values for $N = 2$ and $N = 3$ overlap for all T .

Table 2: LAN end-to-end latency (including network transmission) and TCP packets for Threshold ML-DSA-44. All costs are presented per party.

(T, N)	(2,6)	(4,6)	(6,6)
Signing (ms)	0.548	18.216	8.683
TCP packets	≈15	≈269	≈137

Table 3: WAN signing end-to-end latency (including network transmission) (in ms) for Threshold ML-DSA-44 across different topologies. L = London, S = Seoul, T = Taipei, V = Virginia.

(T, N)	Locations	Signing (ms)
(2,6)	T – S	27.34
(2,6)	T – V	620.43
(4,6)	T – V – L – L	750.65
(6,6)	T – V – L – L – S – S	659.55

Korea). Messages were sent via public IPs and in parallel according to best practices in distributed systems.

The measured signing latencies for both Threshold ML-DSA are within milliseconds, even in WAN setting over multiple continents, demonstrating our schemes’ practicality in real-world distributed systems. While our scheme requires three rounds for a signing operation, these rounds are not executed sequentially in a blocking manner. Instead, messages are sent and processed in parallel across all parties. As a result, the observed latency reflects the most extended communication delay in each round’s critical path—usually between the physically furthest machines—rather than the *sum* of point-to-point delays. Our costs are well under a second, far below a typical TLS handshake timeout of 10–20s [10].

References

- [1] Jenny Blessing, Michael A. Specter, and Daniel J. Weitzner. 2024. Cryptography in the Wild: An Empirical Analysis of Vulnerabilities in Cryptographic Libraries. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) (ASIA CCS '24). Association for Computing Machinery, New York, NY, USA, 605–620. doi:10.1145/3634737.3657012
- [2] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. 2024. Partially Non-interactive Two-Round Lattice-Based Threshold Signatures. In *ASIACRYPT 2024, Part IV (LNCS, Vol. 15487)*, Kai-Min Chung and Yu Sasaki (Eds.). Springer, Singapore, 268–302. doi:10.1007/978-981-96-0894-2_9
- [3] Daniele Cozzo and Nigel P. Smart. 2019. Sharing the LUOV: Threshold Post-quantum Signatures. In *17th IMA International Conference on Cryptography and Coding (LNCS, Vol. 11929)*, Martin Albrecht (Ed.). Springer, Cham, 128–153. doi:10.1007/978-3-030-35199-1_7
- [4] Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. 2025. Simple and Efficient Lattice Threshold Signatures with Identifiable Aborts. *Cryptology ePrint Archive*, Paper 2025/871. <https://eprint.iacr.org/2025/871>
- [5] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. 2024. Threshold Raccoon: Practical Threshold Signatures from Standard Lattice Assumptions. In *EUROCRYPT 2024, Part II (LNCS, Vol. 14652)*, Marc Joye and Gregor Leander (Eds.). Springer, Cham, 219–248. doi:10.1007/978-3-031-58723-8_8
- [6] Rafael del Pino and Guilhem Niot. 2025. Finally! A Compact Lattice-Based Threshold Signature. In *Public-Key Cryptography – PKC 2025*, Tibor Jager and Jiaxin Pan (Eds.). Springer Nature Switzerland, Cham, 169–199.
- [7] Thomas Espitau, Guilhem Niot, and Thomas Prest. 2024. Flood and Submerge: Distributed Key Generation and Robust Threshold Signature from Lattices. In *CRYPTO 2024, Part VII (LNCS, Vol. 14926)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 425–458. doi:10.1007/978-3-031-68394-7_14
- [8] Armando Faz-Hernandez and Kris Kwiatkowski. 2019. *Introducing CIRCL: An Advanced Cryptographic Library*. Cloudflare. Available at <https://github.com/cloudflare/circl>. v1.6.0 Accessed Jan, 2025.
- [9] Kamil Doruk Gür, Jonathan Katz, and Tjerdand Silde. 2024. Two-Round Threshold Lattice-Based Signatures from Threshold Homomorphic Encryption. In *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, Markku-Juhani Saarinen and Daniel Smith-Tone (Eds.). Springer, Cham, 266–300. doi:10.1007/978-3-031-62746-0_12
- [10] IBM. 2023. Handshake timer. <https://www.ibm.com/docs/en/zos/3.1.0?topic=considerations-handshake-timer>. Accessed 06/02/24..
- [11] Wenqing Li, Shijie Jia, Limin Liu, Fangyu Zheng, Yuan Ma, and Jingqiang Lin. 2022. CryptoGo: Automatic Detection of Go Cryptographic API Misuses. In *Proceedings of the 38th Annual Computer Security Applications Conference* (Austin, TX, USA) (ACSAC '22). Association for Computing Machinery, New York, NY, USA, 318–331. doi:10.1145/3564625.3567989
- [12] Dennis Trautwein, Moritz Schubotz, and Bela Gipp. 2021. Introducing Peer Copy - A Fully Decentralized Peer-to-Peer File Transfer Tool. In *2021 IFIP Networking Conference (IFIP Networking)*. 1–2. doi:10.23919/IFIPNetworking52078.2021.9472842